

Lecture 5: Gradient Descent and Linear Regression - Solutions

Overview

This lecture covers gradient-based optimization algorithms and their application to linear regression, fundamental building blocks for training neural networks.

Problem 8: Linear Regression Parameter Optimization

Problem: Consider the linear regression problem where:

- $X \in \mathbb{R}^{m \times d}$: matrix containing training points
- $v \in \mathbb{R}^m$: j th mean-centered feature for all points
- $y \in \mathbb{R}^m$: mean-centered target values
- Loss function: $f(a,b) = \|av + b1 - y\|_2^2$
- $1 \in \mathbb{R}^m$: vector of all ones

Find the optimal value of parameter 'a' that minimizes $f(a,b)$.

Solution:

Step 1: Expand the loss function

$$\begin{aligned} f(a,b) &= \|av + b1 - y\|_2^2 \\ &= (av + b1 - y)^T(av + b1 - y) \end{aligned}$$

Expanding the inner product:

$$f(a,b) = (av)^T(av) + (b1)^T(b1) + y^Ty + 2(av)^T(b1) - 2(av)^Ty - 2(b1)^Ty$$

Step 2: Simplify using vector properties

- $(av)^T(av) = a^2 v^Tv = a^2 \|v\|_2^2$
- $(b1)^T(b1) = b^2 1^T1 = b^2 m$
- $(av)^T(b1) = ab v^T1$
- $(av)^Ty = a v^Ty$
- $(b1)^Ty = b 1^Ty$

Therefore:

$$f(a,b) = a^2 \|v\|_2^2 + b^2 m + \|y\|_2^2 + 2ab v^T1 - 2a v^Ty - 2b 1^Ty$$

Step 3: Take partial derivative with respect to 'a'

$$\partial f / \partial a = 2a \|v\|_2^2 + 2b v^T1 - 2 v^Ty$$

Step 4: Set derivative equal to zero

$$2a \|v\|_2^2 + 2b v^T 1 - 2 v^T y = 0$$

$$a \|v\|_2^2 + b v^T 1 = v^T y$$

Step 5: Solve for optimal 'a'

$$a^* = (v^T y - b v^T 1) / \|v\|_2^2$$

Step 6: Apply mean-centered property

Since v is mean-centered: $v^T 1 = \sum_i v_i = 0$

Since y is mean-centered: $1^T y = \sum_i y_i = 0$

Therefore, the optimal 'a' simplifies to:

$$a = v^T y / \|v\|_2^2$$

This is the standard least-squares solution for a single feature!

Numerical Example:

For $v = [-1, 0, 1]$ and $y = [-2, 0, 2]$:

- $v^T y = (-1)(-2) + (0)(0) + (1)(2) = 4$
- $\|v\|_2^2 = 1 + 0 + 1 = 2$
- $a^* = 4/2 = 2$

Key Concepts Explained**1. Gradient Descent Algorithm**

The basic gradient descent update rule:

$$\theta_{t+1} = \theta_t - \alpha \nabla f(\theta_t)$$

Where:

- θ : parameters to optimize
- α : learning rate (step size)
- $\nabla f(\theta)$: gradient of the objective function

2. Linear Regression Formulation

Matrix form: $y = X\beta + \varepsilon$

- X : design matrix ($m \times d$)
- β : parameter vector ($d \times 1$)
- y : target vector ($m \times 1$)
- ε : noise term

Loss function: $L(\beta) = \|X\beta - y\|_2^2$

Closed-form solution: $\beta^* = (X^T X)^{-1} X^T y$

3. Mean-Centered Data

When features and targets are mean-centered:

- $\sum_i x_i = 0$ (features sum to zero)
- $\sum_i y_i = 0$ (targets sum to zero)
- Simplifies optimization (no bias term needed)

4. Gradient Computation

For loss $f(\theta) = \|X\theta - y\|_2^2$:

$$\nabla f(\theta) = 2X^T X \theta - 2X^T y$$

Setting $\nabla f(\theta) = 0$ gives the normal equations:

$$X^T X \theta = X^T y$$

Gradient Descent Variants

1. Batch Gradient Descent

Uses entire dataset for each update:

$$\theta_{t+1} = \theta_t - \alpha \nabla f(\theta_t)$$

Pros: Stable convergence, exact gradients

Cons: Computationally expensive for large datasets

2. Stochastic Gradient Descent (SGD)

Uses single sample for each update:

$$\theta_{t+1} = \theta_t - \alpha \nabla f_i(\theta_t)$$

Pros: Fast updates, can escape local minima

Cons: Noisy convergence, requires learning rate scheduling

3. Mini-batch Gradient Descent

Uses small batch of samples:

$$\theta_{t+1} = \theta_t - \alpha (1/B) \sum_{i \in \text{batch}} \nabla f_i(\theta_t)$$

Pros: Balance between stability and efficiency

Cons: Requires tuning batch size

Sample Practice Problems

Sample Problem 1

Implement gradient descent for linear regression:

Given $X = \begin{bmatrix} 1 & 2 \\ 3 & 4 \\ 5 & 6 \end{bmatrix}$ and $y = [3, 7, 11]$

1. Initialize parameters $\beta = [0, 0]$
2. Compute gradients analytically
3. Update parameters for 10 iterations with $\alpha = 0.01$

Sample Problem 2

Derive the gradient for regularized linear regression:

$$L(\beta) = \|X\beta - y\|_2^2 + \lambda \|\beta\|_2^2$$

Show that the solution is $\beta^* = (X^T X + \lambda I)^{-1} X^T y$

Sample Problem 3

For polynomial regression with features $[1, x, x^2]$:

1. Set up the design matrix for data points $(1,2), (2,3), (3,6)$
2. Find the least-squares solution
3. Compare with gradient descent convergence

Sample Problem 4

Analyze convergence of gradient descent:

For $f(x) = (1/2)x^T A x - b^T x$ where A is positive definite

1. Show that the optimal learning rate is $2/(\lambda_{\max} + \lambda_{\min})$
2. Prove linear convergence with rate $(\kappa-1)/(\kappa+1)$ where $\kappa = \lambda_{\max}/\lambda_{\min}$

Sample Problem 5

Implement momentum gradient descent:

$$v_{t+1} = \beta v_t + \alpha \nabla f(\theta_t)$$

$$\theta_{t+1} = \theta_t - v_{t+1}$$

Compare convergence with standard gradient descent.

Mathematical Prerequisites

- Multivariable calculus (gradients, partial derivatives)
- Linear algebra (matrix operations, eigenvalues)
- Optimization theory (convexity, convergence analysis)
- Statistics (least squares, maximum likelihood)

- Vector calculus (chain rule, matrix derivatives)